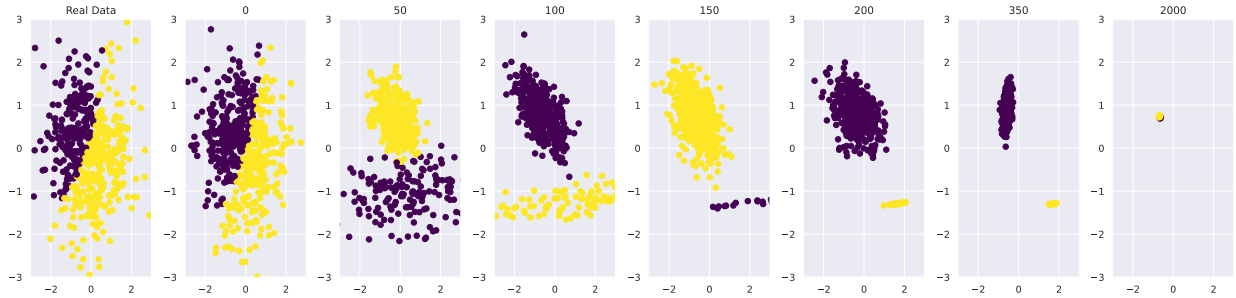
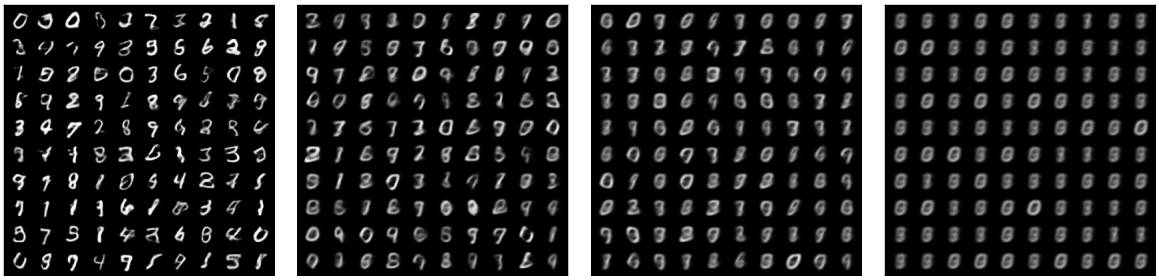


Figure 7: An examples of GMM fitting data at iterations $\{0, 50, 100, 150, 200, 350, 2000\}$. At first the model fits data very well as is shown on the left; yet even at generation 50 the perception of the underlying distribution completely changes. At generation 2000 it converges to a state with very little variance. GMM is sampled a thousand times.



(a) Original model (b) Generation 5 (c) Generation 10 (d) Generation 20

Figure 9: Random latent reconstructions from VAEs. No training data comes from the original distribution. Over the generations, different modes of the original distribution get entangled and generated data starts looking unimodal.

Variational Autoencoders. In this subsection we turn to Variational Autoencoders (VAE). As before, we train an autoencoder on an original data source, which we later sample. Here, we generate latents from a Gaussian distribution which are then used by the decoder to generate data for the subsequent generation. Figure 9 on the left shows an example of generated data using the setting described by Kingma and Welling.

Having performed the process a number of times we arrive at a representation that has very little resemblance of the original classes learned from data. On the right, one sees the generated images from generation 20, which appear to be a mix of all of the different digits. Interestingly, the original encoder perceives the generated data from its descendant with ever-growing confidence – the encoder places such data closer and closer to the mean. Figure 8 shows the density of the latent representation of the original model when presented with data generated by its descendants. As with single-dimensional Gaussians, tails disappear over time and all of the density shifts towards the mean.

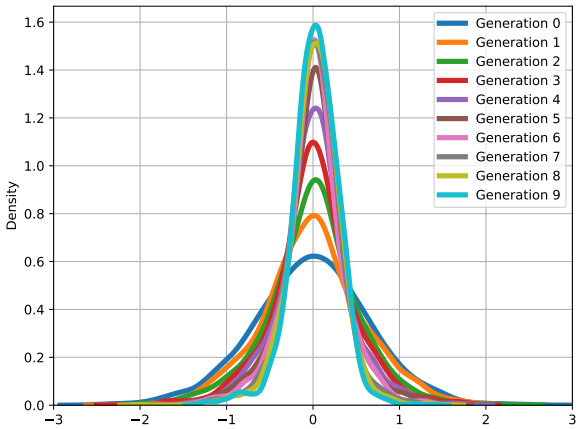
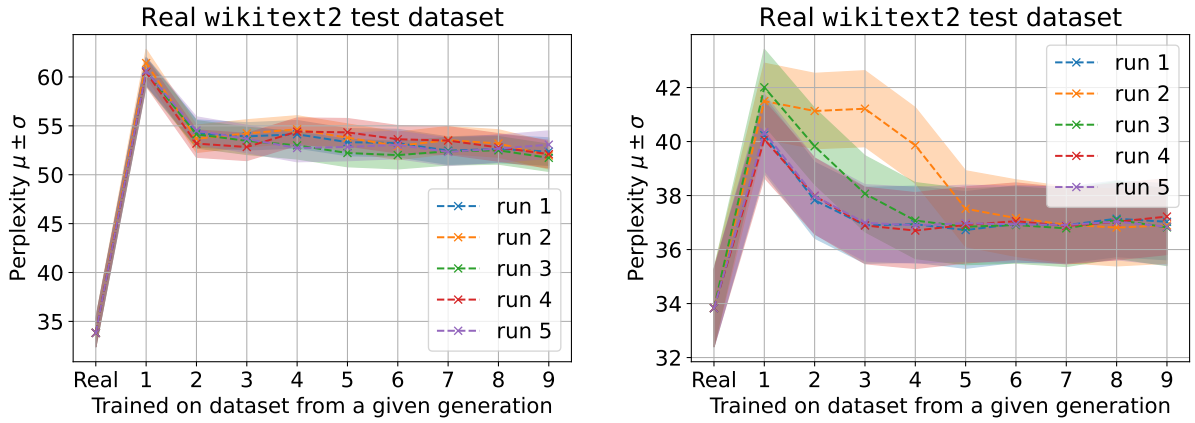


Figure 8: Changing distribution of latents over the learning process with generated data as perceived by the original encoder. Just as with the Gaussian case described above, the tails get washed away and the model arrives at the mean representation of the underlying data.

5.2 Language Models

By now it is clear that *Model Collapse* is universal across different families of ML models. Yet if small models such as GMMs and VAEs are normally trained from scratch, LLMs are different. They are so expensive to retrain from scratch that they are typically initialised with pre-trained



(a) No data preserved, 5 epochs

(b) 10% data preserved, 10 epochs

Figure 10: Performance of OPT-125m models of different generations evaluated using the original `wikitext2` test dataset. Perplexity is shown on the y-axis and for each independent run the graph of the mean and its standard deviation is shown with error bars. x-axis refers to the generation of the model – 'Real' refers to the 'model 0' trained on the original `wikitext2` dataset; model 1 was trained on the data produced by model 0; model 2 was trained on data produced by model 1 etc. with all generated datasets equal in size. We find that models trained on generated data are able to learn some of the original task, but with errors, as seen from the increase in perplexity.

models such as `BERT` [Devlin et al., 2018], `RoBERTa` [Liu et al., 2019], or `GPT2` [Brown et al., 2020], which are trained on large text corpora. They are then fine-tuned to various downstream tasks [Bommasani et al., 2022].

In this subsection we explore what happens with language models when they are sequentially fine-tuned with data generated by other models[4]. We evaluate the most common setting of training a language model – a fine-tuning setting where each of the training cycles starts from a pre-trained model with recent data. Data here comes from another fine-tuned pre-trained model. Since training is restricted to produce models that are close to the original pre-trained model and datapoints generated by the models will generally produce very small gradients, the expectation here may be that the model should only change moderately after fine-tuning. We fine-tune the `OPT-125m` causal language model made available by Meta through `Huggingface` [Zhang et al., 2022].

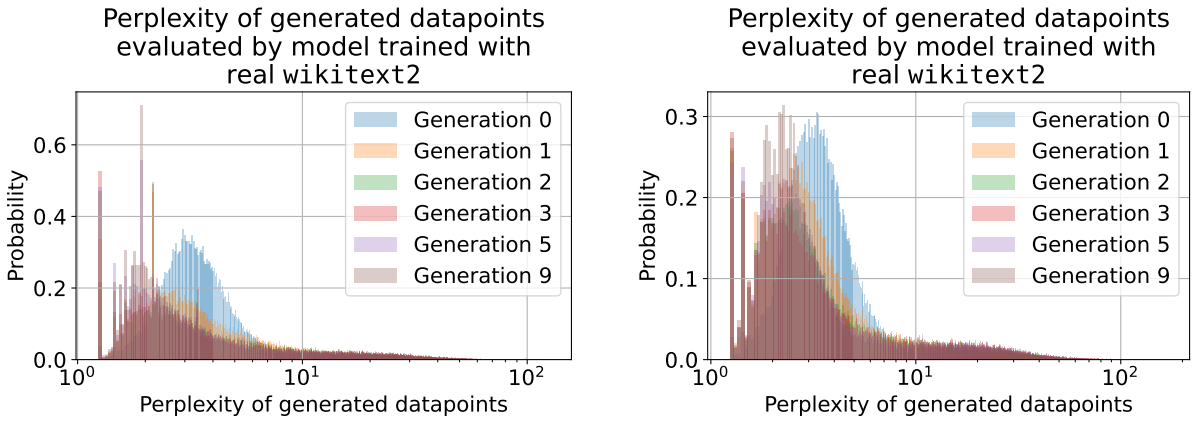
We fine-tune the model on the `wikitext2` dataset. For data generation from the trained models we use a 5-way beam-search. We block training sequences to be 64 tokens long; then for each token sequence in the training set, we ask the model to predict the next 64 tokens. We go through all of the original training dataset and produce an artificial dataset of the same size. Since we go though all of the original dataset and predict all of the blocks, if the model had 0.0 error it would produce the original `wikitext2` dataset. Training for each of the generations starts with generation from the original training data. Each experiment is ran 5 times and the results are shown as 5 separate runs. The original model fine-tuned with real `wikitext2` data gets 34 mean perplexity, from the zero-shot baseline of 115, *i.e.* it successfully learns the task. Finally, to be as realistic as possible, we use the best performing model on the original task, evaluated using the original `wikitext2` validation set, as the base model for the subsequent generations, meaning in practice observed *Model Collapse* can be even more pronounced.

Here we consider two different settings:

5 epochs, no original training data – Here, the model is trained for 5 epochs on the original dataset and no original data. The overall original task performance is presented in Figure 10.(a). We find that training with generated data allows one to adapt to the underlying task, losing some performance – from 20 to 28 perplexity points.

10 epochs, 10% of original training data preserved – Here the model is trained for 10 epochs on the original dataset and every new generation of training, a random 10% of the original data points are sampled. The overall original

[4]One can easily replicate an experiment described in Section 5.1 with a language model to demonstrate *model collapse*. Given that training a single moderately large model produces twice the American lifetime worth of CO_2 [Strubell et al., 2019], we opted to not run such an experiment and instead focus on a more realistic setting for a proof-of-concept. Note that just the language experiments described in the paper took weeks to run.



(a) No data preserved

(b) 10% data preserved

Figure 11: Histograms of perplexities of each individual data training sequence produced by different generations as is evaluated by the very first model trained with the real data. Over the generations models tend to produce samples that the original model trained with real data is more likely to produce. At the same time, a much longer tail appears for later generations – later generations start producing samples that would never be produced by the original model *i.e.* they start misperceiving reality based on errors introduced by their ancestors. Same plots are shown in 3D in Figure 15.

task performance is presented in Figure 10.(b). We find that preservation of the original data allows for better model fine-tuning and leads to only minor degradation of performance.

Both training regimes lead to degraded performance in our models, yet we do find that learning with generated data is possible and models can successfully learn (some of) the underlying task. We now turn to consider the underlying perception of probable events for each generation of our models.

Figure 11 shows histograms of individual datapoint perplexities generated by the models of different generations as is evaluated by the first model developed with real `wikitext2` training data. Here over the generations models tend to produce more sequences that the original model would produce with the higher likelihood. The observed effect is similar to that described for VAEs and GMMs in Section 5.1, where over the generations models started to produce samples that would be produced with higher probabilities by the original model. At the same time, we discover that generated data has much longer tails, suggesting that some of the data would never be produced by the original model – these are the errors that accumulate because of the *learning with generational data*.

We find that data generated by language models in our experiments end up containing a large number of repeating phrases. The repeating problem has been observed in nearly all text generation models [Keskar et al., 2019, Shumailov et al., 2021] and to rule this out as the cause of *Model Collapse*, we further provide numerical experiments when models are explicitly encouraged to produce non-repeating sequences with repeating penalty of 2.0. We find that this causes the models to produce lower score continuations to avoid using repeats, which as a result causes the consequent models to perform even worse. Figure 14 show model perplexities shift across the generations towards more probable token sequences. In particular, enforcing this for the LLM experiments causes the perplexity to double, compared to the original. Models remain as susceptible to *Model Collapse*, if not more.

The described process demonstrates that fine-tuning of language models does not curb the effects of *Model Collapse* and models that are being fine-tuned are also vulnerable. We find that over the generations models tend to produce more probable sequences from the original data and start introducing their own improbable sequences *i.e.* errors.

6 Discussion and Conclusion

We now discuss the implications of *Model Collapse* on the underlying learning dynamics of LLMs. Long-term poisoning attacks on language models are not new. For example, we saw the creation of *click, content, and troll* farms – a form of human 'language models', whose job is to misguide social networks and search algorithms. The negative effect these poisoning attacks had on search results led to changes in search algorithms: *e.g.*, Google downgraded